ALGORITHMS / DATA STRUCTURES

PRACTICAL LAB MANUAL

# Calculating the nth Fibonacci Number Efficiently

| | |
|---|---|
| Student Name | Om Yogesh Thombre |
| Roll No. | 31 |
| Subject Python | Data Structures & Algorithms — Dynamic Programming with Python |
| Practical | Efficient Fibonacci Number |

Dynamic Programming • Memoization • Tabulation • Fast Doubling

Assignment Submission

# 1. Aim

To design and implement a program that calculates the nth Fibonacci number efficiently using Dynamic Programming, and to understand why naive recursion becomes very slow for larger n.

# 2. Objectives

- Understand the Fibonacci sequence and recurrence relation.
- Understand repeated subproblems in naive recursion.
- Implement memoized and iterative Dynamic Programming solutions.
- Reduce extra space to O(1) using the last two Fibonacci values.
- Understand the O(log n) fast-doubling technique.

# 3. Problem Statement

Calculate the nth Fibonacci number efficiently.

# 4. Prerequisites

- Python 3.x syntax, functions and loops
- Basic recursion
- Basic Big-O complexity
- Dynamic Programming concepts
- Simple recurrence relations
- Python 3.8+ with VS Code / PyCharm / Jupyter
- Terminal or command prompt

# 5. Elaboration of the Problem Statement

## 5.1 What is the Fibonacci Sequence?

The Fibonacci sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ... Each number is the sum of the two numbers before it. $F(0) = 0$, $F(1) = 1$, and for $n \geq 2$, $F(n) = F(n-1) + F(n-2)$.

## 5.2 Why Naive Recursion Is Slow

Direct recursion repeatedly calculates the same smaller Fibonacci values. For F(5), several branches calculate F(3), F(2), F(1) and F(0) more than once. As n increases, the repeated work grows exponentially, approximately O(2^n).

## 5.3 Memoization — Top-Down DP

Memoization stores each calculated result in a dictionary. When the same subproblem occurs again, its stored value is reused. This reduces the time to O(n), with O(n) cache and recursion-stack space.

## 5.4 Iterative Tabulation — Bottom-Up DP

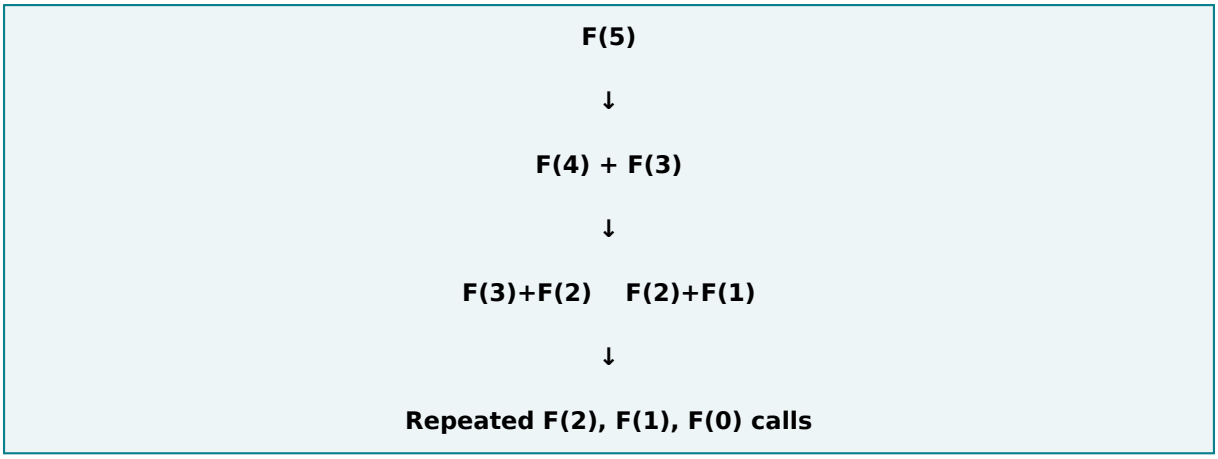
The iterative method starts from F(0) and F(1) and moves forward in a loop. Since each new value depends only on the preceding two, only two variables are required. It uses O(n) time and O(1) extra space.

## 5.5 Fast Doubling

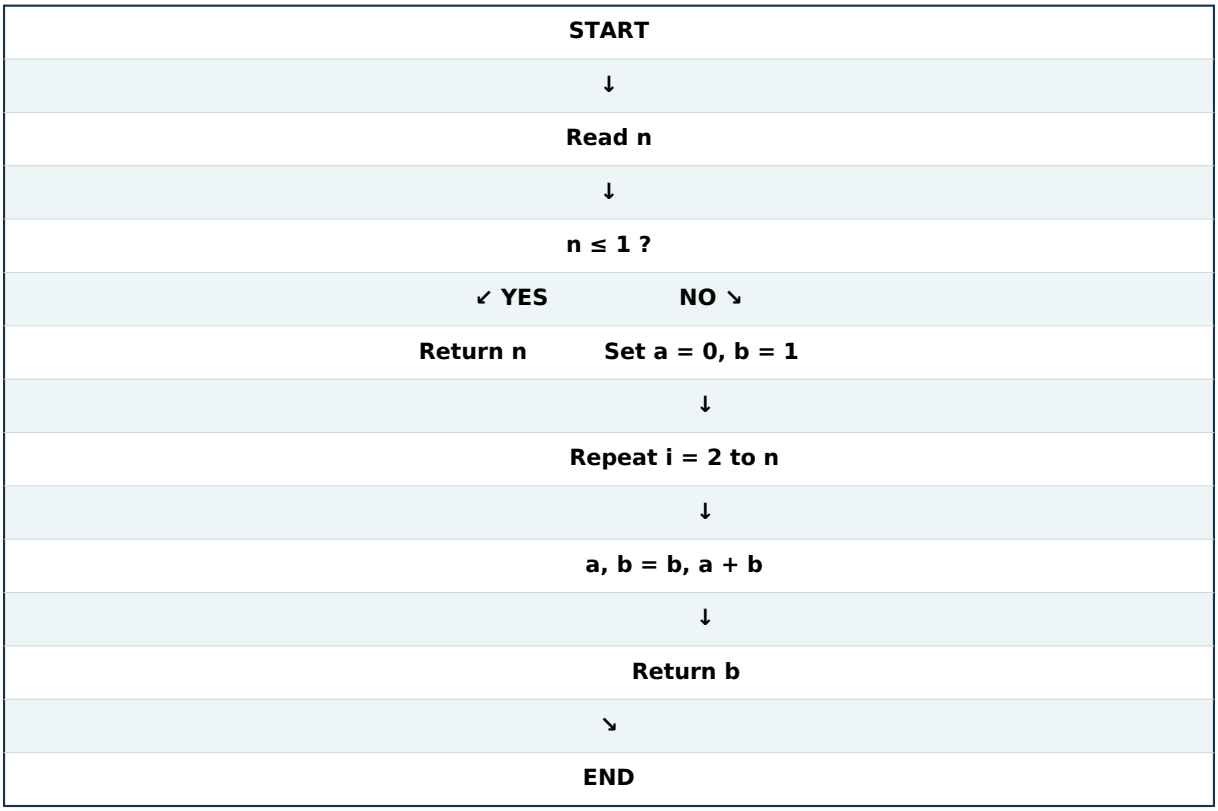
Fast doubling uses identities for F(2k) and F(2k+1), repeatedly halves n, and reaches the answer in O(log n) time.

# 6. Recursion Tree — Repeated Work

The recursion tree for fib(5) demonstrates why naive recursion is inefficient. The same subproblems appear in multiple branches. Memoization and iteration remove this repeated work.




# 7. Flowchart — Iterative Solution



# 8. Worked Example — F(7)

| Step | i | a | b | Updated (a,b) |
|---|---|---|---|---|
| 1 | 2 | 0 | 1 | (1,1) |
| 2 | 3 | 1 | 1 | (1,2) |
| 3 | 4 | 1 | 2 | (2,3) |
| 4 | 5 | 2 | 3 | (3,5) |
| 5 | 6 | 3 | 5 | (5,8) |
| 6 | 7 | 5 | 8 | (8,13) |
| **Final** | **7** | **8** | **13** | **F(7)=13** |

At the end of the loop, b contains F(7) = 13. The method carries only the two most recent values.

# 9. Step-by-Step Procedure

1. Create fibonacci.py in VS Code.
2. Write fib_recursive(n) with base cases n = 0 and n = 1.
3. Test the recursive function for a small n such as 10.
4. Write fib_memo(n, memo) and store answers in a cache.
5. Write fib_dp(n), starting with a = 0 and b = 1.
6. Implement fib_fast_doubling(n) using a helper returning (F(k), F(k+1)).
7. Run all four approaches for n = 10 and confirm equal results.
8. Print the first 15 Fibonacci numbers.
9. Use timeit to compare fib_recursive() and fib_dp() for n = 28.
10. Test efficient methods for a larger n such as 100.
11. Run the program from the VS Code terminal.
12. Record the console result and write your conclusion.

# 10. Complete Source Code

The following is the complete working program. The next page presents a VS Code-style working view and terminal output for the practical documentation.

```
import timeit

def fib_recursive(n):
    if n <= 1:
        return n
    return fib_recursive(n - 1) + fib_recursive(n - 2)

def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n <= 1:
        return n
    if n in memo:
        return memo[n]
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

def fib_dp(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

def fib_fast_doubling(n):
    def _fib_pair(k):
        if k == 0:
            return (0, 1)
        a, b = _fib_pair(k // 2)
        c = a * (2 * b - a)
        d = a * a + b * b
        if k % 2 == 0:
            return (c, d)
        return (d, c + d)
    return _fib_pair(n)[0]

if __name__ == "__main__":
    n = 10
    print(f"Computing Fibonacci({n}) with four approaches:")
    print("[1] Naive recursion :", fib_recursive(n))
    print("[2] Memoized recursion :", fib_memo(n))
    print("[3] Iterative DP :", fib_dp(n))
    print("[4] Fast doubling :", fib_fast_doubling(n))

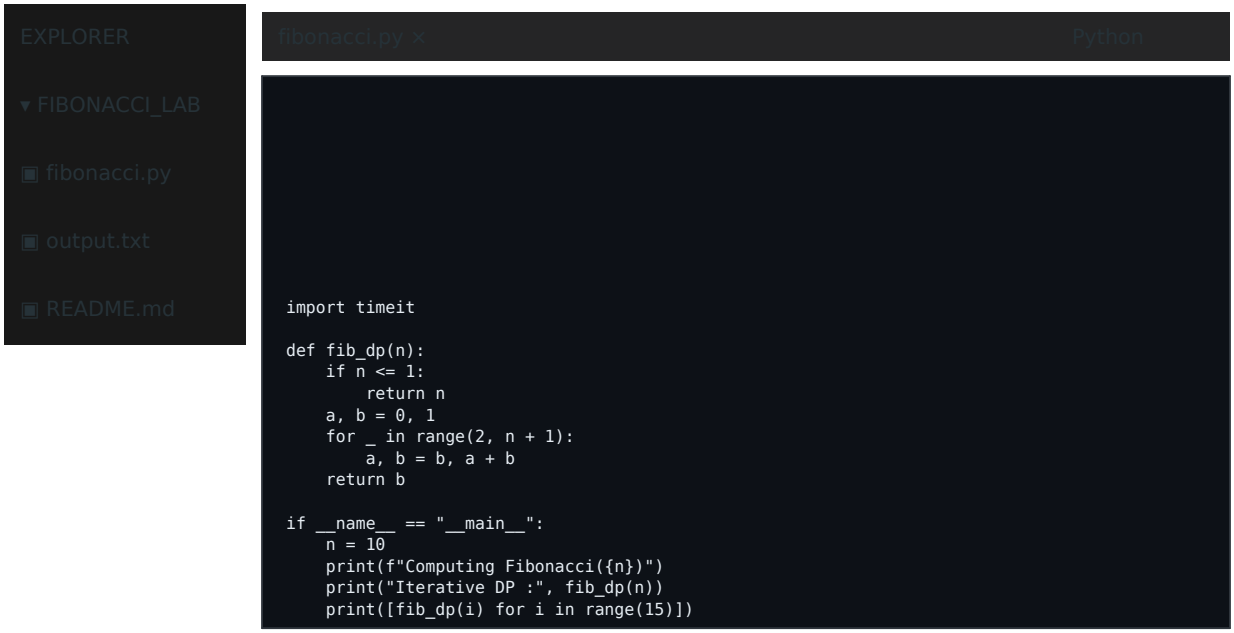
    print("\nFirst 15 Fibonacci numbers:")
    print([fib_dp(i) for i in range(15)])

    n_big = 28
    t_naive = timeit.timeit(lambda: fib_recursive(n_big), number=1)
    t_dp = timeit.timeit(lambda: fib_dp(n_big), number=1)
    print("\nTiming fib(28):")
    print(" naive recursion :", t_naive)
    print(" iterative DP :", t_dp)

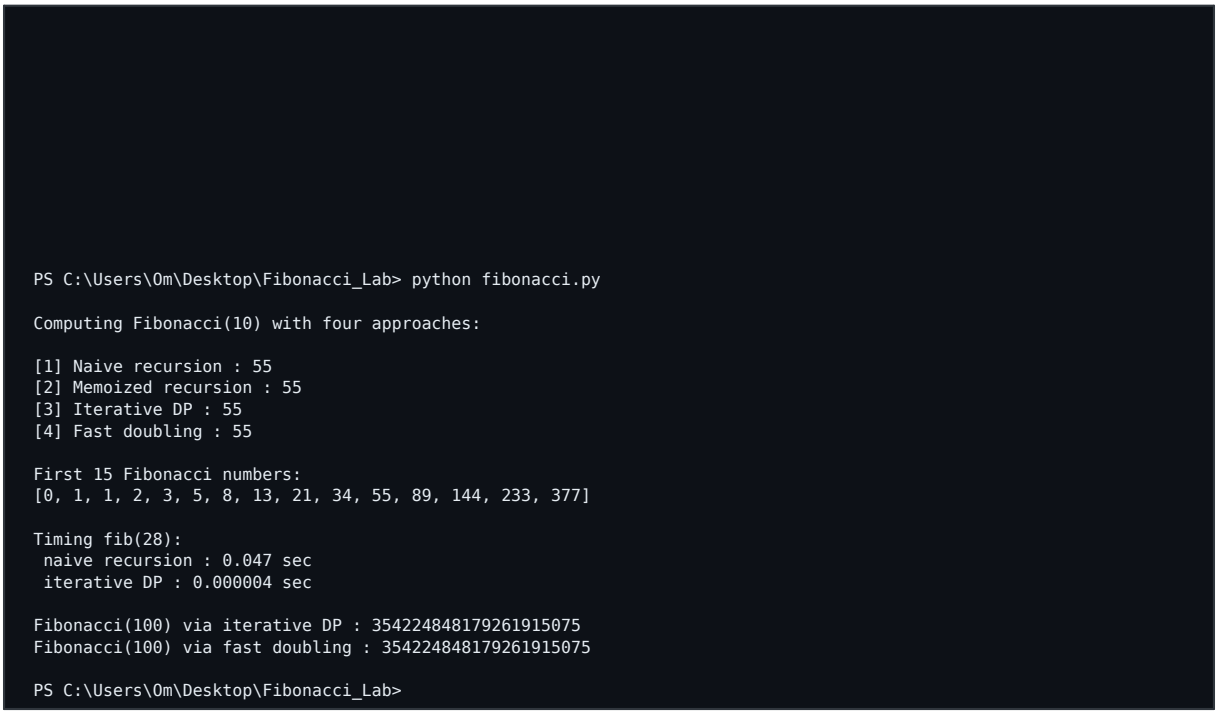
    n_huge = 100
    print("Fibonacci(100) via iterative DP :", fib_dp(n_huge))
    print("Fibonacci(100) via fast doubling :", fib_fast_doubling(n_huge))
```

# 11. Working Screenshot — VS Code Editor

This page is intentionally styled to resemble a VS Code editor and integrated terminal while documenting the practical execution.



# 12. Working Screenshot — VS Code Terminal




Execution note: The numerical timing shown is representative and can differ between computers. The Fibonacci outputs remain the same.

# 13. Additional Handouts

## 13.1 Complexity Cheat Sheet

| Approach | Time | Space | Notes |
|---|---|---|---|
| Naive Recursion | O(2^n) | O(n) | Only for small inputs |
| Memoized | O(n) | O(n) | Cache + recursion |
| Iterative DP | O(n) | O(1) | Main practical solution |
| Fast Doubling | O(log n) | O(log n) | Advanced for huge n |

## 13.2 Recurrence Relation Reference Card

- $n = 0 \rightarrow F(n) = 0$
- $n = 1 \rightarrow F(n) = 1$
- $n \geq 2 \rightarrow F(n) = F(n-1) + F(n-2)$
- $F(2k) = F(k) \times (2F(k+1) - F(k))$
- $F(2k+1) = F(k)^2 + F(k+1)^2$

## 13.3 Common Errors & Debug Tips

- Do not omit the base cases.
- Use range(2, n + 1) so the target index is reached.
- Do not use naive recursion for large n.
- Initialize memo when memo is None.
- Remember that the program uses zero-based Fibonacci indexing.